

PHASE 1 — FOUNDATIONS

Week 2

Prompt Engineering

Reliable instructions = reliable outputs

Zero-shot prompting

Few-shot design

System prompts

Chain-of-thought

Structured output

IT ticket classifier

Week 2 at a Glance

Prompt Patterns

- Anatomy of a good prompt
- Zero-shot vs few-shot
- System vs user messages
- Ambiguity reduction techniques

Applied

- Chain-of-thought prompting
- Structured / JSON output
- IT ticket classifier lab
- Qualitative evaluation



Notebook with 3 real-world prompts + IT ticket classifier with few-shot examples & qualitative eval on ≥ 10 cases

Anatomy of a Good Prompt

Every reliable prompt has five components. Missing any one causes unpredictable output.



Instruction

What the AI must do.
Be verb-specific.

1



Context

Background the
model needs but you
already know.

2



Format

Shape of output:
JSON, bullet list,
table, prose.

3



Constraints

Boundaries: word
count, language, no
hallucination.

4



Examples

Few-shot samples
that show, not just
tell.

5

✗ **Bad:** "Summarise this incident."

✓ **Good:**

"Summarise the incident in 3 bullets: [WHAT] [IMPACT]
[ACTION]. Max 60 words. Audience: on-call manager."

Zero-Shot vs Few-Shot

Use zero-shot for open tasks. Add examples when the format or boundary matters.

Zero-Shot

Use when:

Open-ended, exploration, creative tasks

Advantages:

- No example overhead
- Faster to write
- Works well for most tasks

⚠ Output format may vary between calls

```
Role + Task + Format  
(no examples)
```

Few-Shot

Use when:

Classification, structured output, strict formatting

Advantages:

- Consistent output shape
- Teaches boundaries implicitly
- Handles ambiguous cases better

⚠ Too many examples → context bloat + cost

```
Role + Task + Examples  
+ Format
```

System vs User Messages

System = stable rules. User = variable input. Keep them separate.

SYSTEM

- Persona & role definition
- Non-negotiable rules
- Output schema
- Refusal behaviour
- Tool/plugin permissions

USER

Variable per request:
The actual task, data, or question from the user.



Structured
Output

IT Analogy

System prompt = the connection string + session config. It sets non-negotiables before any query runs.

User message = the SQL query itself. It changes every time; the session config doesn't.

Chain-of-Thought (CoT)

Ask the model to reason first, then answer. Trades latency for accuracy on complex tasks.



Question

Complex IT triage
or multi-step decision



Reasoning

"Let me think step
by step..."



Answer

Accurate, grounded
conclusion



When CoT helps

- Incident root-cause analysis
- Multi-step configuration decisions
- Evaluating trade-offs (vendor, architecture)
- Debugging a logic error



When CoT hurts

- Simple extraction (name, date, ID)
- Single-label classification
- Short yes/no answers
- High-volume, cost-sensitive calls

Structured Output Control

Tell the model exactly what shape to return. Parse it in code. Never screen-scrape.

Prompt instruction:

"Respond ONLY in JSON. No prose. No markdown fences. Exact schema:"

```
{  
  "category": "access | network |  
              hardware | app",  
  "confidence": "low | med | high",  
  "reason": "one sentence",  
  "needs_human": true | false  
}
```



Parse Failure Pattern

- 1 Call LLM with JSON instruction
- 2 Try `json.loads(response)`
- 3 On `ValueError` → retry once with: "Your response was not valid JSON. Return only JSON."
- 4 On second fail → log and raise exception



IT Analogy

Structured output is the API contract. You define the schema; the model implements it. Accepting prose and screen-scraping is like depending on HTML formatting instead of a REST response. Never do it.

Lab: IT Ticket Classifier

Build a few-shot classifier. Evaluate on labeled tickets. Document failure modes.



Classifier Spec

Input:	Ticket title + body (string)
Categories:	access, network, hardware, app
Output:	<pre>{"category": "...", "confidence": "low med high", "reason": "..."} </pre>
Few-shot:	2-4 examples covering boundary cases
Stretch:	Add "needs_human": true when confidence = low



Qualitative Eval (in-class)

- Take 10 labeled tickets (prepared by instructor)
- Run each through your classifier prompt
- Count correct categories → simple accuracy %
- Identify 2 failure modes: mis-classified ticket + low-confidence ticket
- Discuss: would adding one more example fix it?



Rubric:

Notebook runs top-to-bottom · 3 distinct real-world prompts documented · Classifier few-shot included · Qualitative eval on ≥10 cases



Diverse examples



Boundary cases



Failure analysis

After Week 2 you will be able to:

Write prompts that work every time.

- ✦ Decompose any task into instruction, context, format, constraints, examples
- ✦ Choose zero-shot or few-shot based on output consistency requirements
- ✦ Use system prompts for stable rules, user messages for variable input
- ✦ Extract structured JSON output and handle parse failures gracefully
- ✦ Build and evaluate a real IT ticket classifier with qualitative metrics

Build. Learn. Go deep — progressively.